One XML for all the world to transform

Using Xalan to achieve application interoperability

By Chris McManaman

RCG Information Technology

Integration Case Study 5

Why use XML to do this? 6

Why use XSLT to do this? 8

What is XML? 10

Purposes of XML 12

XSL 13

XSLT 14

XSLT Templates 15

Xalan 16

Xalan features 17

Xalan Architecture 18

Xalan command line 20

Xalan debugging options

21

Xalan Core

23

Processing model

24

Location & Step Patterns

26

Patterns

27

Expressions

28

Functions

29

Literal result fragments

32

Recursive processing

35

Creating the XML stream values

36

Conditional processing

37

Sorting

39

Variables and parameters

41

Parameters

_____ **43**

Accessing Java from XSLT

_____ **44**

XSLT Facilitation

_____ **45**

Integration Case Study

The client performed had several batch integrations that were performed during the early hours of the evening. These batch jobs were prone to failure, which would result in several very early morning conference calls to find the problem. This meant that the information from one application to another was now going to be 48 hours late, because the batch job would need to be re-run the following evening.

These late night batch jobs required staff to be available during midnight hours to monitor the process.

Applications that were not integrated required a lot of manually duplicate data entry and coordination between the departments to maintain data quality. Every quarter they would perform a manual audit of the data to make sure it was in synch.

The objective for the company was to:

- Create a real time global company.
- Increase data quality between business processes.
- Reduce the number of employees working during the evenings.

Why use XML to do this?

We decided on using XML and web services.

XML enabled us:

- The ability to create units of work
  - This way if transactions failed it would be on an individual transaction basis.
  - An error in a single record could fail an entire batch process.
- To send one XML that could be transformed and validated using XSLT
  - One XML could sent to and formatted by multiple applications
  - Target applications had control of how they wanted the data to look
  - Validation could be performed on the data before being processed by the target application
- Create information with context
  - The XML described business objects more precisely than a flat file through the use of parent/child elements and attributes
  - Information in XML is more searchable and easier for developers to work with.

Web services enabled us to:

- Send data in real time.
  - This allowed developers to analyze and resend the transactions with seconds of an error
  - Data was consistent between applications making departmental and customer communications significantly more effective
  - The business process was real time rather than 24
- Integrate a heterogeneous environment into an ERP.

- Reduced the need for the ERP to be the central repository for all corporate information

- Applications that were previously incompatible

- Future purchases of applications could be dictated by the needs of the business process rather than what was ‘out of the box’ compatible with existing applications.

❖ For performance, we compressed the XML into binary zip files.

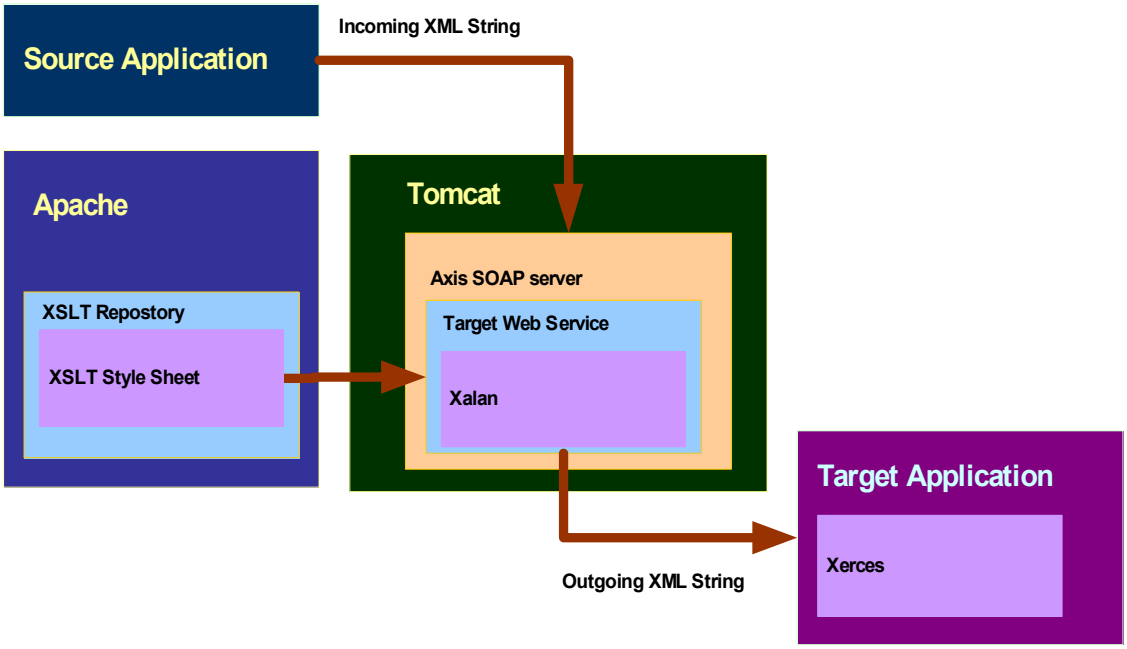
- This prevented the need to control the size of XML’s being sent across the middleware.

Why use XSLT to do this?

One of the great enablers of the web services revolution will be XSLT. Sending XML from one application to another is far simpler when you have the ability to strip and customize the incoming XML to the format that the target application requires.

As mentioned, XSLT allows one application to send an XML stream to several target applications. Each application will have its own requirements of how the data should be formatted. XSLT allows the target application developers to control this formatting and validation. Any changes to in their application can be transferred to the incoming XSLT file.

An XSLT file is easier to update and publish than making changes to code and recompiling. We built a central XSLT repository on Apache which allowed developers publish and edit XSLT files on the fly. An xml string would come into the target web service and the web service would apply the corresponding xslt to the xml string. The resulting xml string would be passed onto the target application.



We did have a middleware to process error handling and emailing. However we quickly learned to take as much out of the middleware as you can for the following reasons:

- People love to blame any transaction problems on the middleware because it was a black box.

- Ownership of the data should exist at the source and target ends.

- Business rules and data manipulation in the middleware requires a significant amount of maintenance.

What is XML?

XML stands for extensible markup language. It is a framework for defining markup languages.

There is no fixed collection of markup tags - we may define our own tags, tailored for our kind of information each XML language is targeted at its own application domain, but the languages will share many features. This allows developers to customize a markup language that fits their corporate data. A corporate Meta data markup language repository would enable developers to come up with a common language to describe business objects that exist within their corporation.

There is a common set of generic tools for processing documents such as SAX, DOM, XPath and XSLT.

- SAX – Simple API for XML
  - Better able to process larger XML documents
  - Does not load up the entire document into memory
  - SAX takes a document and pulls it through the processor looking for elements that trigger events.
- DOM – Document Object Model
  - Allows developers to use XPATH to search the document and go directly to the information you need
  - DOM requires the entire XML document to be loaded into memory
  - Most developers find DOM easier to use
- XPath
  - A query language for XML documents.
- XSLT
  - Allows developers to transforms documents from one type to another.

- XLink
  - Enables developers to define links between XML documents.
- XPointer
  - Used to point to document fragments within XML documents

Purposes of XML

XML allows developers to separate **syntax** from **semantics** to provide a common framework for structuring information (browser rendering semantics is completely defined by style sheets). XML allows **tailor-made markup** for any imaginable application domain. This makes it very useful for coming up with a common markup language for a corporations business entities.

There is also support **internationalization** (Unicode) and **platform independence.** The platform independence is incredibly useful for integration projects in large companies. Most companies contain a wide mixture of mainframe, client server and web applications that are in house or out of the box and developed in several different languages.

XML may be the future of structured information, including **databases,** because of its ability to give meaning to the information. This meaning enables developers to have a better understanding for the purpose of the data. XML has a better ability to represent the object oriented data than a relation database and flat files.

XSL

XSL (e**X**tensible **S**tylesheet **L**anguage) consists of two parts:

- *XSL Transformations* (**XSLT**)
- *XSL Formatting Objects* (**XSL-FO**)

XSLT

You can use Xalan in conjunction with an XSLT style sheet to transform XML documents.

A transformation expressed in XSLT describes rules for transforming a source tree into a result tree.

A pattern is matched against elements in the source tree. A template is instantiated to create part of the result tree.

XSLT Templates

- When a template is instantiated, each instruction is executed and replaced by the result tree fragment that it creates.
- Elements are only processed when they have been selected by the execution of an instruction.
- The result tree is constructed by finding the template rule for the root node and instantiating its template.
- When a template is instantiated, each instruction is executed and replaced by the result tree fragment that it creates.
- Elements are only processed when they have been selected by the execution of an instruction.
- The result tree is constructed by finding the template rule for the root node and instantiating its template.

Xalan

Xalan-Java is an XSLT processor for transforming XML documents into other XML document types. We used it to format and validate incoming XML Strings. However, you can use XSLT to transform XML to html.

Xalan contains the XSL transformation vocabulary and XPath.

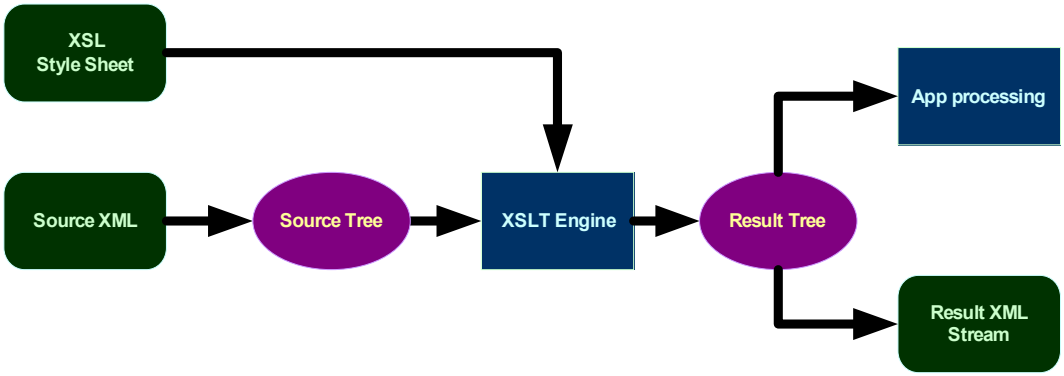
It can be used from the command line, or as a module in another program. We used the command line version of Xalan to develop the XSLT style sheets. Xalan was even adopted by the .net developers because of its advanced ability to provide debugging information during the development. We incorporated Xalan into the target web services to transform and validate the XML strings from the source applications.

Xalan features

- Includes an Interpretive processor for use in a tooling and debugging environment and a Compiling processor (XSLTC) for use in a high performance runtime environment.
- Implements the relevant W3C specifications: XSL Transformations (XSLT) Version 1.0 and XML Path Language (XPath) Version 1.0.
- Implements TrAX (Transformation API for XML), now part of Java API for XML Processing 1.2, and builds on SAX 2 and DOM level 2.
- May be configured to work with any XML parser, such as Xerces-Java, that implements Java API for XML Processing 1.2 (see Plugging in an XML parser).
- Can process Stream, SAX or DOM input, and output to a Stream, SAX or DOM.
- Transformations may be chained (the output of one transformation may be the input for another).
- May be run from the command line for convenient file-to-file transformations.
- Supports the creation of Java and scripting language extensions.
- And provides a growing library of extension elements and functions.

Xalan Architecture

The following diagram shows the XSLT abstract processing model. A transformation expressed in XSLT describes rules for transforming a Source Tree into a result tree. Xalan acts as the XSLT engine.



Basic steps

The following java class takes an xml file and transforms it into a different xml file using a XSLT stylesheet.

The objects that inherit the Javax.xml.transform.source interface are used to build the source input and stylesheet.

The objects that inherit the Javax.xml.transform.result interface are used to build the result tree.

TransformerFactory creates a Transformer which is used to transformer object with the stylesheet. The transformer object can then create the result tree by passing the source xml name and the output xml name to the transform method.

```
import java.io.File;

import javax.xml.transform.Transformer;
import javax.xml.transform.TransformerFactory;
import javax.xml.transform.stream.StreamResult;
import javax.xml.transform.stream.StreamSource;

class XSLTProcessor
{

  public static void main(String args[]) throws javax.xml.transform.TransformerException
  {
    if (args.length != 2)
    {
      System.err.println("Usage:");
      System.err.println(" java " + Sample1.class.getName()
        + " xmlFilename xsltFileName");
      System.exit(1);
    }


    File xmlFile = new File("c://xslt_pres//xml_files//incoming.xml");
    File xsltFile = new File("c://xslt_pres//xslt_files//elements.xsl");

    javax.xml.transform.Source xmlSource =  new StreamSource(xmlFile);

    javax.xml.transform.Source xsltSource = new StreamSource(xsltFile);

    javax.xml.transform.Result result = new StreamResult(System.out);

    TransformerFactory transFact = TransformerFactory.newInstance();

    Transformer trans = transFact.newTransformer(xsltSource);

    trans.transform(xmlSource, result);
  }

}
```

Xalan command line

The following script allows you to pass a debugging option (%1), an input xml file (%2), an XSLT stylesheet (%3) and an output XML file (%4).

We used variations of this script to develop the XSLT stylesheets.

```
java -cp .;C:\xslt_pres\xalan\bin\bsf.jar;
            C:\xslt_pres\xalan\bin\xalan.jar;
            C:\xslt_pres\xalan\bin\xercesImpl.jar;
            C:\xslt_pres\xalan\bin\xml-apis.jar;
            C:\xslt_pres\xalan\bin\xsltc.jar
            org.apache.xalan.xslt.Process %1 -in %2 -xsl %3 -out %4
```

Xalan debugging options

- XSLTC
  - Use xsltc for transformation
- IN
  - input xml
- XSL
  - Location of the xslt style sheet
- OUT
  - Name of the output file
- TT
  - Trace the templates as they are being called. The example shows that a pattern in the incoming xml matches the ‘root/daily’ template.
  - file:///C:/xslt_pres/in.xsl Line #3, Column #28: template match='root/daily’
- TG
  - Trace each generation event. This shows you the elements that are being constructed in the result tree. The following example shows you that the <well_number> element will contain the value 90003039 in the resulting XML file.

    STARTDOCUMENT
    STARTELEMENT: well_number
    CHARACTERS: 90003039
    ENDELEMENT: well_number
    ENDDOCUMENT

- TS
  - Trace each selection event. Good for seeing what the boolean value was of a test.

    file:///C:/xslt_pres/in.xsl Line #8, Column #46: if, test='starts-with(well_no,'9')': true

- TTC
  - Trace the template children as they are being processed.

    - template match=’/’
    - apply-templates
    - template match='root/daily’
    - element
    - if
    - value-of

Xalan Core

- org.apache.xalan.processor
  - The module that processes the stylesheet, and provides the main entry point into Xalan.
- org.apache.xalan.templates
  - The module that defines the stylesheet structures, including the Stylesheet object, template element instructions, and Attribute Value Templates.
- org.apache.xalan.transformer
  - The module that applies the source tree to the Templates, and produces a result tree.
- org.apache.xpath
  - The module that processes both XPath expressions, and XSLT Match patterns.

Processing model

- An XSLT stylesheet consists of a number of **template rules**:
  - **template rule = pattern + template**
- An XML document will be processed through XSLT by :
  - The XML will be processed starting at the root node and working its way down the document to construct a tree.
  - A single **node** is processed by:
    - **finding** the template rule with the best matching pattern

      The following template rule:

      <xsl:apply-templates select="root/daily" />

      matches:

      <root>
      <daily>match</daily>
      </root>

    - **instantiating** its template (creates result fragment + continues processing recursively)
      - **after the first template is executed the XSLT processor will go on to the next template to see if there are any matches.**

The following XSLT style sheet shows that each time you find the parent/child node of <root><daily> apply the “root/daily” template.

The *<xsl:template match="/">* statement is the parent template that starts at the top most node of the source tree xml.

```
<xsl:stylesheet version="1.0"
    xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

  <xsl:template match="/">
        <xsl:apply-templates select="root/daily" />
  </xsl:template>

  <xsl:template match="root/daily">
    <xsl:element name="well_number">
        <xsl:if test="starts-with(well_no,'9')">
                <xsl:value-of select="well_no" />
        </xsl:if>
    </xsl:element>
  </xsl:template>
</xsl:stylesheet>
```

Incoming.xml

```
<?xml version='1.0' encoding='windows-1252'?>
<root>
  <daily>
    <well_name country='ca'>Calgary North</well_name>
    <well_no>494995-4040</well_no>
    <gas>
      <volume>939</volume>
      <uom>mmbtu</uom>
      <ownership>15</ownership>
    </gas>
  </daily>
</root>
```

Location & Step Patterns

- '/' defines:
  - a parent child relationship
  - A separator between step patterns
- Root/daily/gas/volume
  - We need to find header elements that are children of the root element.
  - "root/daily/gas/volume" will not find "root/daily/oil/volume" elements.
- Root//volume
  - Find <volume> elements that are nested to any level.
  - "root//volume" will find both gas and oil volumes

```
<?xml version='1.0' encoding='windows-1252'?>
<root>
  <daily>
    <well_name country='ca'>Calgary North</well_name>
    <well_no>494995-4040</well_no>
    <gas>
      <volume>939</volume>
      <uom>mmbtu</uom>
      <ownership>15</ownership>
    </gas>
    <oil>
      <volume>334</volume>
      <uom>mmbtu</uom>
      <ownership>4</ownership>
    </oil>
  </daily>
</root>
```

Patterns

- We traverse the **source tree** to find **node-sets** that ***match a pattern*** determine by an **XPath expression**

```
<xsl:if test="volume > 1000">

<xsl:if test="starts-with(well_no,'494995')">
```

- A given node **matches** a given pattern if:
  - the node is member of the result of **evaluating** the pattern with respect to **some** context.
  - Operationally (and intuitively), a pattern matching is probably best evaluated backwards (from right to left).

Expressions

Types:

- **node-set** (set of nodes)
- **boolean** (true or false)
- **number** (floating point)
- **string** (Unicode text)

An expression can be:

- a constant, e.g. **"..."**
- a variable: **$*variable***
- a function call: ***function* (*arguments*)**
- a boolean expression: **or**, **and**, **=**, **!=**, **<**, **>**, **<=**, **>=** (standard precedence, all left associative)
- a numerical expression: **+**, **-**, *, **div**, **mod**
- a node-set expression (using **location paths**!): **|** (set union)

Functions

- Manipulating Numbers
  - Floor()
  - Ceiling()
  - Round()
  - Format-number(number,format)
- Manipulating Strings
  - Substring-before(string,location)
  - Substring-after(string,location)
  - Substring(string, location)
  - Concat(value,value,…)
  - String-length(string)

function.xsl

```
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <!-- Root template -->
  <xsl:template match="/">
      <xsl:apply-templates select="root//daily" />
  </xsl:template>
  <xsl:template match="root//daily">
                <xsl:if test="starts-with(well_no,'4')">
        <xsl:element name="well_name">
        <xsl:value-of select="substring-after(well_name,' ')" />
        </xsl:element>
        <xsl:element name="well_number">
          <xsl:value-of select="well_no" />
        </xsl:element>
        <xsl:element name="city_country">
          <xsl:value-of select="concat(substring-before(well_name,' '),'-
',well_name/@country)" />
        </xsl:element>
        <xsl:element name="gas_volume">
          <xsl:value-of select="floor(gas/volume)" />
        </xsl:element>
        <xsl:element name="oil_volume">
          <xsl:value-of select="ceiling(oil/volume)" />
        </xsl:element>
        <xsl:element name="liquid_volume">
          <xsl:value-of select="round(liquids/volume)" />
        </xsl:element>
        </xsl:if>
  </xsl:template>
</xsl:stylesheet>
```

Literal result fragments

- **<xsl:text ...> ... </...>** (as raw text, but with white-space and character escaping control)
- **<xsl:comment> ... </...>** (inserts a comment **<!--...-->**)

Since literal fragments are part of the stylesheet XML document, only well-formed XML will be generated.

```
<xsl:stylesheet version="1.0"
    xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <!-- Root template -->
  <xsl:template match="/">
      <xsl:apply-templates select="root/daily" />
  </xsl:template>
  <xsl:template match="root/daily">
          <xsl:element name="well_number">
              <xsl:value-of select="well_no" />
          </xsl:element>
          <xsl:comment>Specify the object type</xsl:comment>
          <xsl:element name="ofs_type">
              <xsl:text>Well</xsl:text>
          </xsl:element>
  </xsl:template>
</xsl:stylesheet>
```

Literal result fragments

- Recursive processing instructions:
  - **<xsl:apply-templates select="*node-set expression*" .../>**
    apply pattern matching and template instantiation on selected nodes (default: all children)
  - **<xsl:call-template name="..."/>**
    invoke template by name (where **xsl:template** has **name="..."** attribute)
  - **<xsl:for-each select="*node-set expression*"> *template* </...>**
    instantiate inlined template for each node in node-set (document order by default)
  - **<xsl:copy> *template* </...>**
    copy current node to output and apply template (shallow copy)
  - **<xsl:copy-of select="..."/>**
    copy selected nodes to output (deep copy, includes descendants)

Recursive.xsl

```
<xsl:stylesheet version="1.0"
    xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <!-- Root template -->
  <xsl:preserve-space elements="/" />
  <xsl:template match="/">
      <xsl:copy-of select="root/daily/oil" />
      <xsl:copy>
        <xsl:apply-templates select="root/daily/gas" />
      </xsl:copy>
      <xsl:apply-templates select="root/daily/liquid" />
  </xsl:template>
  <xsl:template match="root/daily/oil">
        <xsl:element name="oil_volume">
            <xsl:value-of select="volume" />
        </xsl:element>
 </xsl:template>
  <xsl:template match="root/daily/gas">
        <xsl:element name="gas_volume">
            <xsl:value-of select="volume" />
        </xsl:element>
 </xsl:template>
  <xsl:template match="root/daily/liquid">
        <xsl:element name="liquid_volume">
            <xsl:value-of select="volume" />
        </xsl:element>
 </xsl:template>
</xsl:stylesheet>
```

calltemplate.xsl

```
<xsl:stylesheet version="1.0"
     xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <!-- Root template -->
  <xsl:template match="/">
      <xsl:for-each select="root/daily">
      <xsl:sort select="well_no"
              order="descending"/>
      <xsl:call-template name="templatecall">
        <xsl:with-param name='wellno' select="well_no"/>
      </xsl:call-template>
      </xsl:for-each>
  </xsl:template>
  <xsl:template name="templatecall">
    <xsl:param name="wellno"/>
          <xsl:element name="well_number">
            <xsl:value-of select="$wellno" />
              </xsl:element>
           <xsl:comment>Specify the object type</xsl:comment>
              <xsl:element name="ofs_type">
            <xsl:text>Well</xsl:text>
              </xsl:element>
  </xsl:template>
</xsl:stylesheet>
```

Recursive processing

- **Processing modes: mode="..."** on **xsl:template** and **xsl:apply-templates** allow an element to be processed multiple times in different ways.

Recursivemode.xsl

```
<xsl:stylesheet version="1.0"
    xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <!-- Root template -->
   <xsl:template match="/">
     <xsl:apply-templates select="root/daily"/>
  </xsl:template>
  <xsl:template match="root/daily">
    <xsl:if test="well_name/@type='us'">
     <xsl:apply-templates select="well_name" mode="ca"/>
    </xsl:if>
    <xsl:if test="well_name/@type='ca'">
     <xsl:apply-templates select="well_name" mode="us"/>
       </xsl:if>
 </xsl:template>
  <xsl:template match="well_name" mode="ca">
                <xsl:element name="canadian_well">
            <xsl:value-of select="." />
                </xsl:element>
 </xsl:template>
  <xsl:template match="well_name" mode="us">
                <xsl:element name="us_well">
            <xsl:value-of select="." />
                </xsl:element>
 </xsl:template>
</xsl:stylesheet>
```

Creating the XML stream values

The following XPath expressions create the resulting xml stream

<xsl:element name="..." namespace="..."> ... </...>
construct an element with the given name, attributes, and contents

<xsl:attribute name="..." namespace="..."> ... </...>
construct an attribute (inside **xsl:element**)

<xsl:value-of select="..."/>
construct character data or attribute value (expression converted to string)

<xsl:processing-instruction name="..."> ... </...>
construct a processing instruction

elements.xsl

```
<xsl:stylesheet version="1.0"
    xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <!-- Root template -->
  <xsl:template match="/">
      <xsl:apply-templates select="root/daily">
        <xsl:sort select="well_number"
                data-type="number"
                order="descending" />
      </xsl:apply-templates>
  </xsl:template>
  <xsl:template match="root/daily">
        <xsl:element name="well_number">
                <xsl:attribute name="well_name" >
                        <xsl:value-of select="well_name" />
                </xsl:attribute>
                 <xsl:value-of select="well_no" />
         </xsl:element>
         <xsl:element name="country">
                 <xsl:value-of select="well_name/@country" />
          </xsl:element>
  </xsl:template>
</xsl:stylesheet>
```

Conditional processing

Processing can be conditional:

- **<xsl:if test="*expression*"> ... </...>**

apply template if expression (coerced to boolean) evaluates to true

- **<xsl:choose>**
  <xsl:when test="*expression*"> ... </...>
  ...
  <xsl:otherwise> ... </...>
  </...>

  - **Choose-when-otherwise is the if - else if - else equivalent**

Choose.xsl

```
<xsl:stylesheet version="1.0"
     xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <!-- Root template -->
   <xsl:template match="/">
      <xsl:apply-templates select="root/daily"/>
  </xsl:template>
  <xsl:template match="root/daily">
     <xsl:choose>
      <xsl:when test="substring-after(well_name,'-') = 'South'">
        <xsl:apply-templates select="well_name" mode="us"/>
      </xsl:when>
      <xsl:when test="substring-after(well_name,'-') = 'North'">
        <xsl:apply-templates select="well_name" mode="ca"/>
      </xsl:when>
      <xsl:otherwise>
        <xsl:apply-templates select="well_name" mode="ae"/>
      </xsl:otherwise>
    </xsl:choose>
 </xsl:template>
  <xsl:template match="well_name" mode="ca">
          <xsl:element name="canadian_well">
                  <xsl:value-of select="." />
          </xsl:element>
 </xsl:template>
  <xsl:template match="well_name" mode="us">
          <xsl:element name="us_well">
               <xsl:value-of select="." />
          </xsl:element>
 </xsl:template>
  <xsl:template match="well_name" mode="ae">
          <xsl:element name="ae_well">
               <xsl:value-of select="." />
          </xsl:element>
 </xsl:template>
</xsl:stylesheet>
```

Sorting

- Sort the order of elements by :
  - **<xsl:sort select="*expression*" .../>**;
    a sequence of **xsl:sort** elements placed in **xsl:apply-templates** or **xsl:for-each** defines a lexicographic order
  - **Sorting attributes**:
    - order="ascending/descending"
    - lang="..."
    - data-type="text/number"
    - case-order="upper-first/lower-first"
- **Sorting can only be used within:**
  - **xsl:apply-templates**
  - **xsl:for-each**

calltemplate.xsl

```
<xsl:stylesheet version="1.0"
     xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <!-- Root template -->
  <xsl:template match="/">
       <xsl:for-each select="root/daily">
       <xsl:sort select="well_no"
                order="descending"/>
       <xsl:call-template name="templatecall">
         <xsl:with-param name='wellno' select="well_no"/>
       </xsl:call-template>
       </xsl:for-each>
  </xsl:template>
  <xsl:template name="templatecall">
     <xsl:param name="wellno"/>
           <xsl:element name="well_number">
              <xsl:value-of select="$wellno" />
                 </xsl:element>
            <xsl:comment>Specify the object type</xsl:comment>
                 <xsl:element name="ofs_type">
              <xsl:text>Well</xsl:text>
                 </xsl:element>
  </xsl:template>
</xsl:stylesheet>
```

Variables and parameters

- for reusing results of computations and parameterizing templates and whole stylesheets
  - static scope rules
  - can hold any XPath value (string, number, boolean, node-set) + **result-tree fragment**
  - purely declarative: variables cannot be updated
  - can be global or local to a template rule

- Declaration:
  - **<xsl:variable name="..." select="*expression*"/>**
    variable declaration, value given by XPath expression
  - **<xsl:variable name="..."> *template* </..>**
    variable declaration, template is instantiated as result tree fragment to give value

- Access the variables using **$<variable_name>**

```
<xsl:stylesheet version="1.0"
    xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <!-- Root template -->
  <xsl:template match="/">
        <xsl:apply-templates select="root/daily" />
      <xsl:apply-templates select="root//gas" />
  </xsl:template>
  <xsl:template match="root/daily">
    <xsl:element name="well_number">
        <xsl:if test="starts-with(well_no,'9')">
                <xsl:value-of select="well_no" />
        </xsl:if>
    </xsl:element>
  </xsl:template>
  <xsl:template match="root//gas">
                <xsl:variable name="percentage" select="ownership" />
        <xsl:if test="boolean(volume > 1000)">
                <xsl:variable name="percentage" select="1" />
                <xsl:element name="percent">
                        <xsl:value-of select="$percentage * 100" />
                </xsl:element>
    </xsl:if>
  </xsl:template>
</xsl:stylesheet>
```

Parameters

- To access and pass variables within a stylesheet use:

  - **$*name***
    returns XPath value in expressions, e.g. attribute value templates

  - **xsl:with-param**
    passes parameters in xsl:call-template and xsl:apply-templates

calltemplate.xsl

```
<xsl:stylesheet version="1.0"
    xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <!-- Root template -->
  <xsl:template match="/">
      <xsl:for-each select="root/daily">
      <xsl:sort select="well_no"
              order="descending"/>
      <xsl:call-template name="templatecall">
        <xsl:with-param name='wellno' select="well_no"/>
      </xsl:call-template>
      </xsl:for-each>
  </xsl:template>
  <xsl:template name="templatecall">
    <xsl:param name="wellno"/>
          <xsl:element name="well_number">
            <xsl:value-of select="$wellno" />
              </xsl:element>
           <xsl:comment>Specify the object type</xsl:comment>
              <xsl:element name="ofs_type">
            <xsl:text>Well</xsl:text>
              </xsl:element>
  </xsl:template>
</xsl:stylesheet>
```

Accessing Java from XSLT

- Create a java function and then include it in your classpath. You need to have a namespace pointing at the package that you want to access in the XSLT stylesheet.

- It is very important to use - exclude-result-prefixes="javafunc“ – otherwise the xmlns:javafunc="java.lang" namespace will be littered all

- xmlns:javafunc="java.lang“
- exclude-result-prefixes="javafunc“

```
<xsl:value-of select="javafunc:System.currentTimeMillis()"/>
```

```
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
                        xmlns:javafunc="java.lang"
                        exclude-result-prefixes="javafunc">
  <!-- Root template -->
  <xsl:template match="/">
      <xsl:for-each select="root/daily">
      <xsl:sort select="well_no"
              order="descending"/>
      <xsl:call-template name="templatecall">
        <xsl:with-param name='wellno' select="well_no"/>
      </xsl:call-template>
      </xsl:for-each>
  </xsl:template>
  <xsl:template name="templatecall">
    <xsl:param name="wellno"/>
          <xsl:element name="well_number">
             <xsl:value-of select="$wellno" />
               </xsl:element>
          <xsl:element name="create_date">
             <xsl:value-of select="javafunc:System.currentTimeMillis()"/>
          </xsl:element>
  </xsl:template>
</xsl:stylesheet>
```

XSLT Facilitation

- Start small

XSLT is an interpretive language that is different from procedural and object oriented coding. Your biggest obstacle in getting companies to embrace XSLT and Xalan is providing lots of training and facilitation for the developers.

In the first months there was a lot of expectation for failure, now the biggest skeptics have embraced XSLT.

- Work with the developers not for them

Facilitation is the training methodology that produced the most success. Consultants who did the XSLT's for the developers experienced problems later on. The consultants now owned all the XSLT's for the application and became overburdened making changes to them. With any project, the number of bugs that are found two weeks before production increases significantly.

Consultants that produced a team of XSLT experts were better able to deal with problems that occurred before and during production.

- Test their XSLT's = Don't let the developers fail

During the first months make sure you test the developers XSLT. We let the developers produce the XSLT style sheets and then we would review them with feedback on better ways to do them. We used the sandwich approach with the developers. "Wow, this is a great first try. I think that you could use call-template rather than apply-template but besides that it looks great!".